Journal of Applied Data Sciences
Vol. 4, No. 4, Juni 2025, pp.
414-430

Sistem Deteksi Emosi pada Teks Menggunakan Metode LSTM

Akbyna Riyomi [1], Cindy Ardelya [2, *], Fatharony Fatwa [3,]

[1,2,4] *Information Technology, Politeknik Caltex Riau, Jalan Umban Sari No.1, Pekanbaru, Riau 28265, Indonesia*

(Received: June 13, 2025; Revised: July 5, 2025; Accepted: July 27, 2025; Available online: August 10, 2025)

Abstract

Perkembangan media sosial telah menjadikan teks sebagai salah satu bentuk utama ekspresi pengguna, termasuk dalam mengungkapkan emosi dan kondisi psikologis seperti stres. Penelitian ini bertujuan untuk mengembangkan sistem deteksi emosi berbasis analisis teks menggunakan model *Long Short-Term Memory (LSTM)*. Dataset yang digunakan dalam penelitian ini diperoleh dari platform Kaggle dan mencakup data teks dengan berbagai label emosi dalam dua bahasa. Proses pengolahan data mencakup tahapan pembersihan, tokenisasi, normalisasi, dan pelabelan emosi. Sistem yang dibangun diharapkan mampu mengidentifikasi emosi dalam teks secara akurat, sehingga dapat diterapkan dalam chatbot konseling atau sistem monitoring psikologis berbasis teks.

Keywords: Deteksi Emosi, Analisis Teks, LSTM.

1. Introduction

Perkembangan teknologi informasi dan komunikasi, khususnya media sosial, telah mengubah cara manusia berinteraksi dan mengekspresikan emosi mereka. Platform seperti Twitter, Facebook, dan Instagram memungkinkan pengguna untuk membagikan pemikiran, pengalaman, serta keluh kesah secara real-time melalui teks. Unggahan-unggahan tersebut tidak hanya berfungsi sebagai sarana komunikasi, tetapi juga dapat merefleksikan kondisi emosional pengguna, seperti perasaan sedih, marah, cemas, bahagia, dan berbagai emosi lainnya.

Fenomena ini membuka peluang penelitian yang besar, khususnya dalam bidang pemrosesan bahasa alami (Natural Language Processing/NLP), untuk membangun sistem yang mampu mendeteksi emosi secara otomatis dari teks yang diunggah pengguna. Sistem semacam ini memiliki potensi untuk diterapkan dalam berbagai bidang, termasuk analisis media sosial, peningkatan layanan pelanggan, serta pengembangan sistem pendukung kesehatan mental. Namun, deteksi emosi berbasis teks bukanlah tugas yang sederhana. Bahasa manusia sangat kompleks, di mana emosi dapat diungkapkan secara eksplisit maupun implisit, dengan penggunaan kata yang memiliki makna ganda, variasi gaya bahasa, serta perbedaan konteks budaya. Oleh karena itu, diperlukan metode yang mampu memahami pola urutan kata dan menangkap konteks secara mendalam.

Long Short-Term Memory (LSTM) merupakan salah satu metode deep learning yang terbukti efektif dalam mengolah data sekuensial seperti teks. LSTM memiliki kemampuan untuk mengingat informasi penting dalam urutan kata, sehingga dapat memahami makna kalimat dengan lebih akurat dibandingkan metode konvensional. Dengan keunggulan tersebut, penelitian ini memilih LSTM sebagai metode utama dalam membangun sistem deteksi emosi pada teks.

Penelitian ini bertujuan untuk mengembangkan sistem deteksi emosi berbasis LSTM dengan memanfaatkan dataset publik yang dikumpulkan dari platform Kaggle. Dataset yang digunakan terdiri dari teks berbahasa Indonesia dan Inggris yang telah diberi label emosi seperti sedih, marah, cemas, bahagia, dan lainnya. Sebelum digunakan pada model, data akan melalui serangkaian proses pra-pemrosesan seperti pembersihan teks, tokenisasi, normalisasi, dan penghapusan kata tidak penting (stopwords).
Dengan sistem yang dikembangkan, diharapkan penelitian ini dapat memberikan kontribusi pada pengembangan teknologi kecerdasan buatan, khususnya dalam analisis emosi berbasis teks, yang bermanfaat untuk penelitian akademis, layanan publik, serta sistem deteksi dini kondisi emosional pengguna.

2. Research Methodology

Berdasarkan uraian sebelumnya, penelitian ini mengeksplorasi domain deteksi emosi pada teks media sosial menggunakan pendekatan Long Short-Term Memory (LSTM), yang terbukti efektif untuk memproses data sekuensial. Penelitian ini memanfaatkan dataset publik yang berisi kumpulan teks berbahasa Indonesia dan Inggris dengan label emosi (happy, anger, sadness, love, dan fear) yang diperoleh dari platform Kaggle.

Metodologi penelitian ini menjelaskan langkah-langkah yang dilakukan untuk membangun sistem deteksi emosi pada teks media sosial menggunakan metode Long Short-Term Memory (LSTM) :

1) Analisis Kebutuhan Penelitian

Tahap ini meliputi penentuan tujuan penelitian, pemilihan dataset, dan identifikasi emosi yang akan diklasifikasikan (happy, anger, sadness, love, fear). Dataset yang digunakan diperoleh dari Kaggle dengan format CSV yang telah memiliki label emosi.

2) Pra-Pemrosesan Data (Text Pre-processing)

Data mentah dari dataset akan dibersihkan dengan langkah-langkah berikut:

- Cleaning: Menghapus URL, mention (@username), hashtag, angka, simbol, dan karakter non-alfabet.
- Case Folding: Mengubah semua huruf menjadi huruf kecil.
- Stopword Removal: Menghapus kata-kata umum yang tidak memiliki makna signifikan.
- Stemming dan Normalisasi: Mengubah kata menjadi bentuk dasar dan mengganti kata gaul (slang) dengan kata baku menggunakan kamus khusus.

Hasil dari tahap ini berupa teks bersih dan siap diubah menjadi representasi numerik.

3) Tokenisasi dan Padding

Setelah pembersihan, teks diubah menjadi deretan angka (token) menggunakan Tokenizer Keras. Untuk memastikan panjang input seragam, diterapkan padding pada setiap urutan kata dengan panjang maksimum 100 kata.

4) Pembangunan Model LSTM

Model dibangun menggunakan TensorFlow dan Keras dengan arsitektur:

- Embedding Layer: input_dim=5000, output_dim=128, input_length=100.
-
- LSTM Layer: 64 unit.
- Dropout Layer: 0.5 untuk mencegah overfitting.
- Dense Layer: 32 unit (aktivasi ReLU).
- Output Layer: Softmax (untuk 5 kelas emosi).

5) Pembagian Data (Data Split)

Dataset dibagi menjadi 80% data latih dan 20% data uji menggunakan metode stratified split untuk menjaga proporsi label.

6) Pelatihan Model (Training)

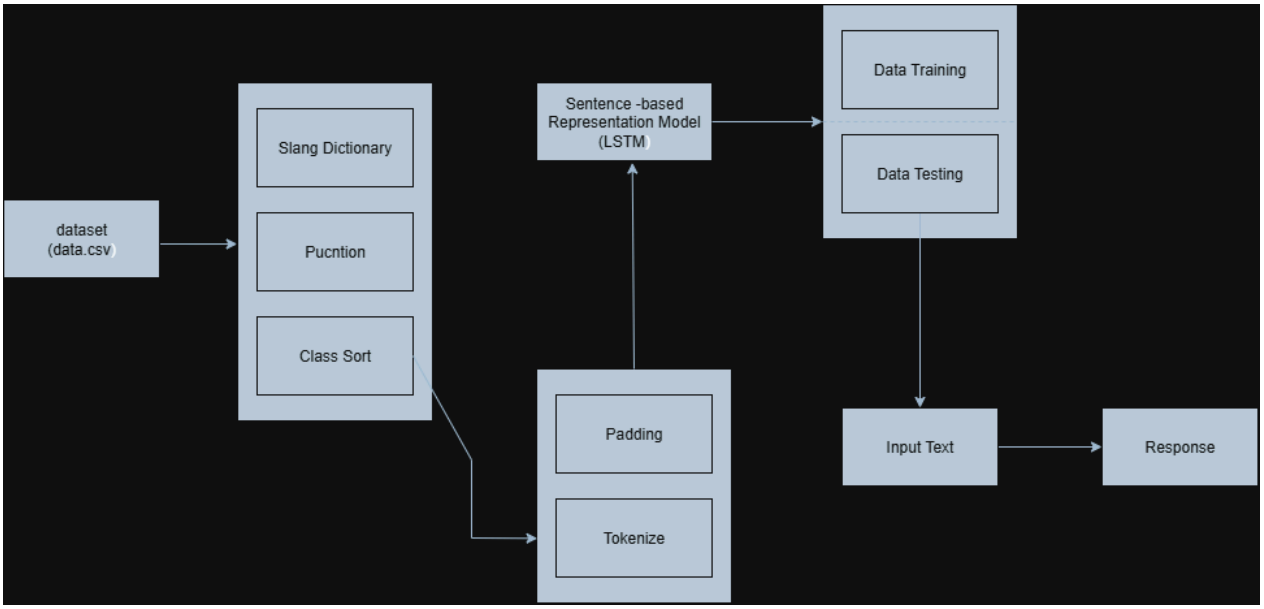
Model LSTM dilatih selama 6 epoch dengan batch size 32 dan optimisasi Adam. Validasi dilakukan pada data uji (X_test, y_test).

7) Evaluasi Model

Evaluasi dilakukan menggunakan metrik akurasi, precision, recall, F1-score, serta confusion matrix. Hasil evaluasi digunakan untuk mengukur performa model dalam mengklasifikasikan emosi.

2.1. Arsitektur Sistem

Arsitektur sistem deteksi emosi yang diusulkan ditunjukkan pada Gambar 2. Alur sistem dapat dijelaskan sebagai berikut:



2.1 Arsitektur Sistem

Adapun penjelasan dari arsitektur yang digunakan pada Gambar di atas adalah sebagai berikut:

1) Proses dimulai dari dataset (data.csv), yaitu sebuah kumpulan data teks yang digunakan sebagai sumber informasi. Dataset ini nantinya akan diproses lebih lanjut sebelum dapat digunakan pada tahapan selanjutnya.
2) Sebelum melangkah ke proses pembelajaran, teks mentah pada dataset akan melalui tahapan preprocessing. Dalam tahapan ini, pertama-tama terjadi proses penghapusan tanda baca (punctuation) untuk membersihkan kalimat dari karakter-karakter yang tidak relevan. Setelah itu, slang dictionary diterapkan untuk menormalkan bahasa gaul atau kata tidak baku menjadi kata baku. Langkah selanjutnya adalah lemmatization, yaitu proses menemukan bentuk kata dasarnya, sehingga dapat menyederhanakan variasi kata yang terjadi. Setelah proses lemmatization, kalimat kemudian dikelompok berdasarkan kelas (Class Sort) sesuai label yang tersedia.
3) Selanjutnya, pada tahapan Tokenize, kalimat yang masih berupa teks diubah menjadi sebuah deret nomor atau indeks. Hal ini berguna agar dapat diproses lebih lanjut oleh model. Setelah proses Tokenize, diterapkan Padding, yaitu menyamakan ukuran kalimat dengan cara menambah padding pada kalimat yang lebih pendek. Dengan begitu, ukuran kalimat menjadi seragam dan siap dimasukkan ke model.
4) Kalimat yang telah diberi padding kemudian diteruskan ke Sentence-based Representation Model (LSTM). Dalam proses ini, LSTM belajar dari urutan kata dan konteks kalimat untuk mendeteksi emosi .
5) Sebelum melatih model, dataset dibagi menjadi dua, yaitu data training dan data testing, demi menjaga kualitas dan validitas model. Setelah proses pelatihan, model yang dihasilkan nantinya dapat digunakan untuk menganalisa kalimat input dan memberikan respon berupa deteksi emosi berdasarkan kalimat yang dimasukkan.

2.1. Dataset

Pengumpulan data merupakan tahap awal dalam pengembangan sistem deteksi emosi berbasis analisis teks. Dalam penelitian ini, data dikumpulkan dari platform terbuka berupa kumpulan teks media sosial, yang diunduh dari Kaggle. Dataset terdiri dari tweet dalam bahasa Indonesia dan Inggris yang telah dilabeli berdasarkan kategori emosi seperti marah, senang, takut, sedih, dan lainnya.

Selain itu, data juga mencakup berbagai ekspresi pengguna yang mencerminkan kondisi emosional atau potensi stres, sehingga dapat dimanfaatkan untuk pelatihan dan pengujian model. Format data telah disusun dalam bentuk *CSV* yang berisi pasangan antara teks (tweet) dan label emosi. Dataset ini kemudian dikonversi ke format yang sesuai untuk pemrosesan lebih lanjut dan pelatihan model, seperti dalam format JSON atau array terstruktur.

label		tweet
anger happy Other (2283)	25% 23% 52%	**4389** unique values
anger		Soal jln Jatibaru,polisi tdk bs GERTAK gubernur .Emangny polisi tdk ikut pmbhasan? Jgn berpolitik. P...

3.1 Dataset Indonesia Twitter

tweet_id		sentiment		content
1.69b 1.97b		neutral worry Other (22903)	22% 21% 57%	**39827** unique values
1956967341		empty		@tiffanylue i know i was listenin to bad habit earlier and i started freakin at his part =[

2.1 Dataset Emotion Detection

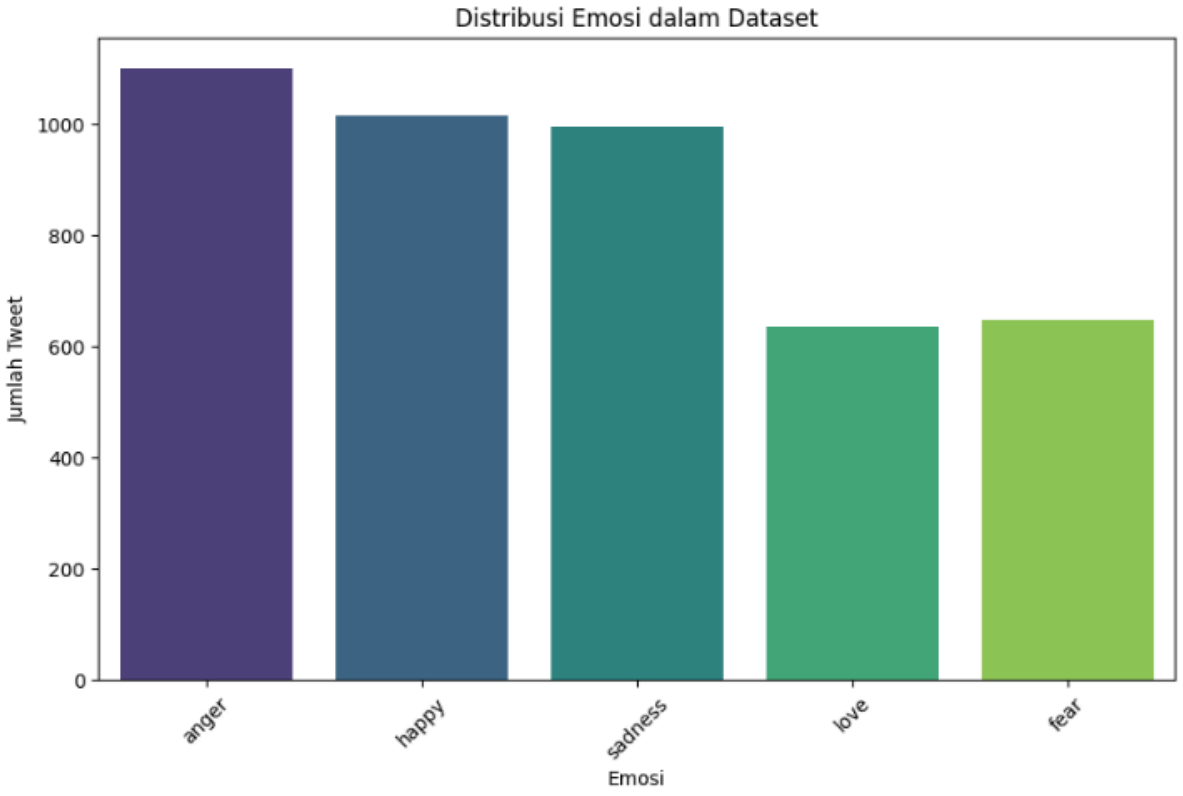
3. Results and Discussions

Bab ini membahas hasil penelitian, meliputi analisis dataset, tahap pra-pemrosesan data, pembangunan serta pelatihan model LSTM, dan evaluasi performa model. Seluruh proses ini dirancang untuk menghasilkan sistem deteksi emosi berbasis analisis teks dengan tingkat akurasi yang optimal.

3.1. Dataset

Dataset yang digunakan dalam penelitian ini diperoleh dari Kaggle, berisi kumpulan tweet berbahasa Indonesia dan Inggris dengan label emosi seperti anger, happy, sadness, love, dan fear. Dataset disimpan dalam format CSV dengan dua kolom utama: tweet (teks) dan label (kategori emosi).

Distribusi jumlah data di setiap label ditunjukkan pada Gambar 1. Dari gambar terlihat bahwa anger memiliki jumlah data terbanyak (lebih dari 1.000 tweet), diikuti oleh happy dan sadness. Sedangkan love dan fear memiliki jumlah data paling sedikit (sekitar 600-an tweet).



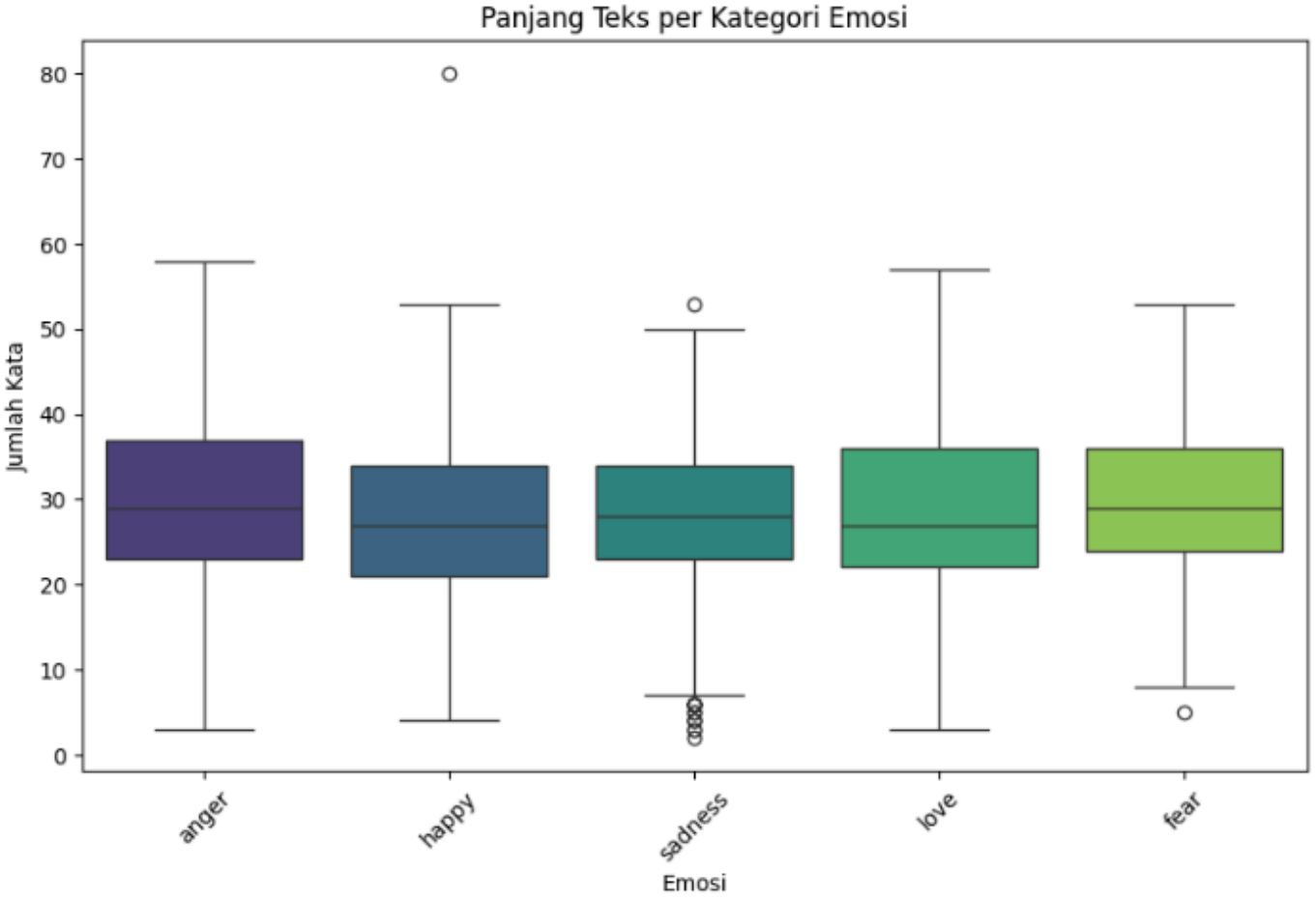
Gambar 3.1 Distribusi Emosi dalam Dataset

3.2. Analisis Panjang Text

Analisis panjang teks dilakukan untuk mengetahui sebaran jumlah kata dalam tweet pada setiap kategori emosi. Hasil visualisasi menggunakan boxplot menunjukkan bahwa:

1) Sebagian besar tweet memiliki panjang antara 20–35 kata.
2) Kategori love memiliki variasi panjang teks yang paling beragam (terlihat dari boxplot dengan rentang yang lebih lebar).
3) Beberapa outlier terdapat pada kategori happy dan sadness, dengan panjang tweet mencapai lebih dari 70 kata.
4) Visualisasi boxplot ditunjukkan pada Gambar 2.

Visualisasi boxplot ditunjukkan pada **Gambar 3.2**.



Gambar 3.2 Distribusi Emosi dalam Dataset

3.3. Text Processing

Tahapan pra-pemrosesan teks dilakukan untuk membersihkan data mentah sehingga siap diproses oleh model.

Kode Cleaning Teks :

Listing 1. Cleaning Teks

```
def clean_text(text):
    text = text.lower()
    text = re.sub(r"http\S+|@\w+|#\w+|\d+|[^\w\s]", "", text)
    text = re.sub(r"\s+", " ", text).strip()
    text = " ".join([word for word in text.split() if word not in stop_words])
    return text

df["clean_text"] = df["tweet"].astype(str).apply(clean_text)
```

3.4. WordCloud

Untuk mengetahui kata-kata dominan di setiap kategori emosi, dilakukan visualisasi WordCloud.

Hasil WordCloud menunjukkan bahwa kata "bahagia" dan "senang" paling sering muncul pada kategori happy, sedangkan kategori anger didominasi kata "kesal" dan "marah".

Kode WordCloud :

Listing 2. WordCloud untuk tiap emosi.

```
def generate_wordcloud(text, title):
    wordcloud = WordCloud(width=800, height=400,
                          background_color='white',
                          stopwords=stop_words).generate(text)
    plt.figure(figsize=(10,5))
    plt.imshow(wordcloud)
    plt.title(title, size=15)
    plt.axis('off')
    plt.show()

for emotion in df['label'].unique():
    text = ' '.join(df[df['label']==emotion]['tweet'].tolist())
    generate_wordcloud(text, f'Kata yang Sering Muncul - Emosi: {emotion}')
```

3.5. Pembangunan Model LSTM

Model dibangun dengan arsitektur sebagai berikut:

- Embedding Layer: input_dim=5000, output_dim=128, input_length=100.
- LSTM Layer: 64 unit.
- Dropout Layer: 0.5.
- Dense Layer: 32 unit (ReLU).
- Output Layer: Softmax (5 kelas emosi).

Kode Model :

Listing 3. Arsitektur LSTM.

```
model = Sequential([
    Embedding(input_dim=5000, output_dim=128, input_length=100),
    LSTM(64, return_sequences=False),
    Dropout(0.5),
    Dense(32, activation='relu'),
    Dense(len(le.classes_), activation='softmax')
```

```
])

model.compile(loss='sparse_categorical_crossentropy',
              optimizer='adam',
              metrics=['accuracy'])
```

3.6. Pelatihan Model

Model dilatih dengan 6 epoch, batch size=32, dan validasi pada data uji.

Listing 4. Proses pelatihan model.

```
model.fit(X_train, y_train,
          epochs=6,
          batch_size=32,
          validation_data=(X_test, y_test))
```

3.7. Evaluasi Model

Evaluasi menggunakan classification report dan confusion matrix.

Listing 5. Evaluasi model.

```
y_pred = model.predict(X_test)
y_pred_classes = np.argmax(y_pred, axis=1)
print(classification_report(y_test, y_pred_classes, target_names=le.classes_))
```

Table 1. Hasil Evaluasi Model LSTM berdasarkan Precision, Recall, dan F1-Score

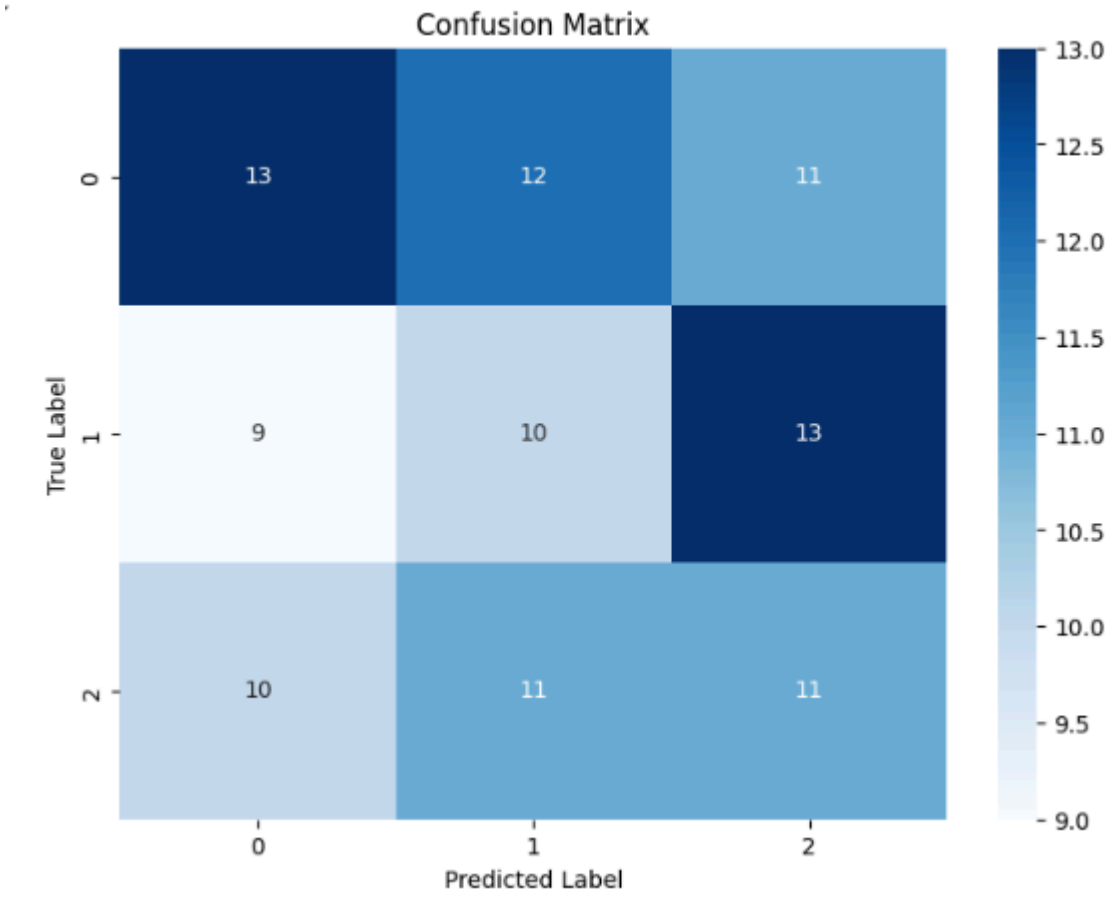
Emosi	Precision	Recall	F1-score	Support
Anger	0.56	0.77	0.65	220
Fear	0.65	0.65	0.65	130
Happy	0.54	0.56	0.55	204
Love	0.75	0.61	0.67	127
Sadness	0.48	0.33	0.39	200

Accuracy	-	-	**0.58**	881
macro avg	0.60	0.58	0.58	881
weighted avg	0.58	0.58	0.57	881

Confusion Matrix:

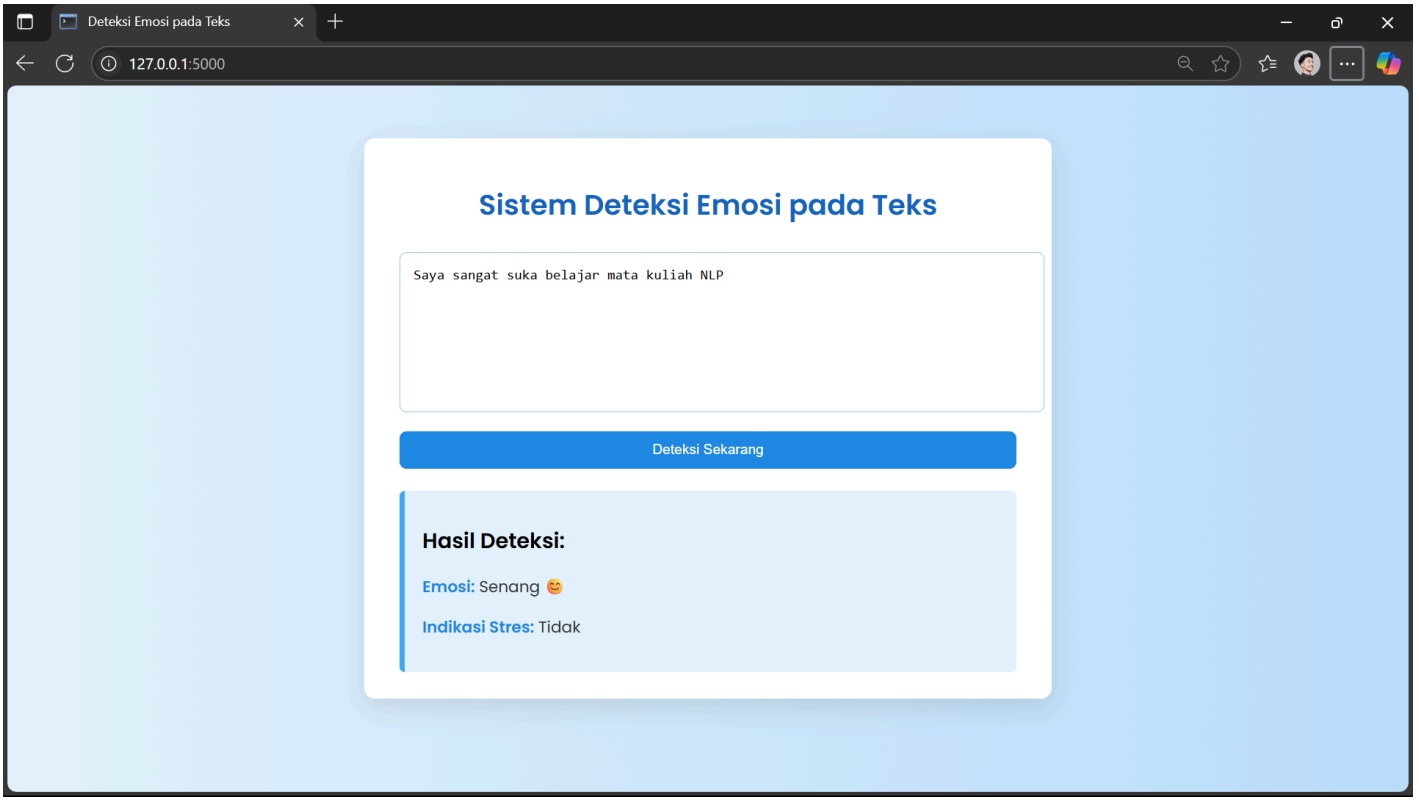
Listing 6. Visualisasi confusion matrix.

```
cm = confusion_matrix(y_test, y_pred_classes)
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

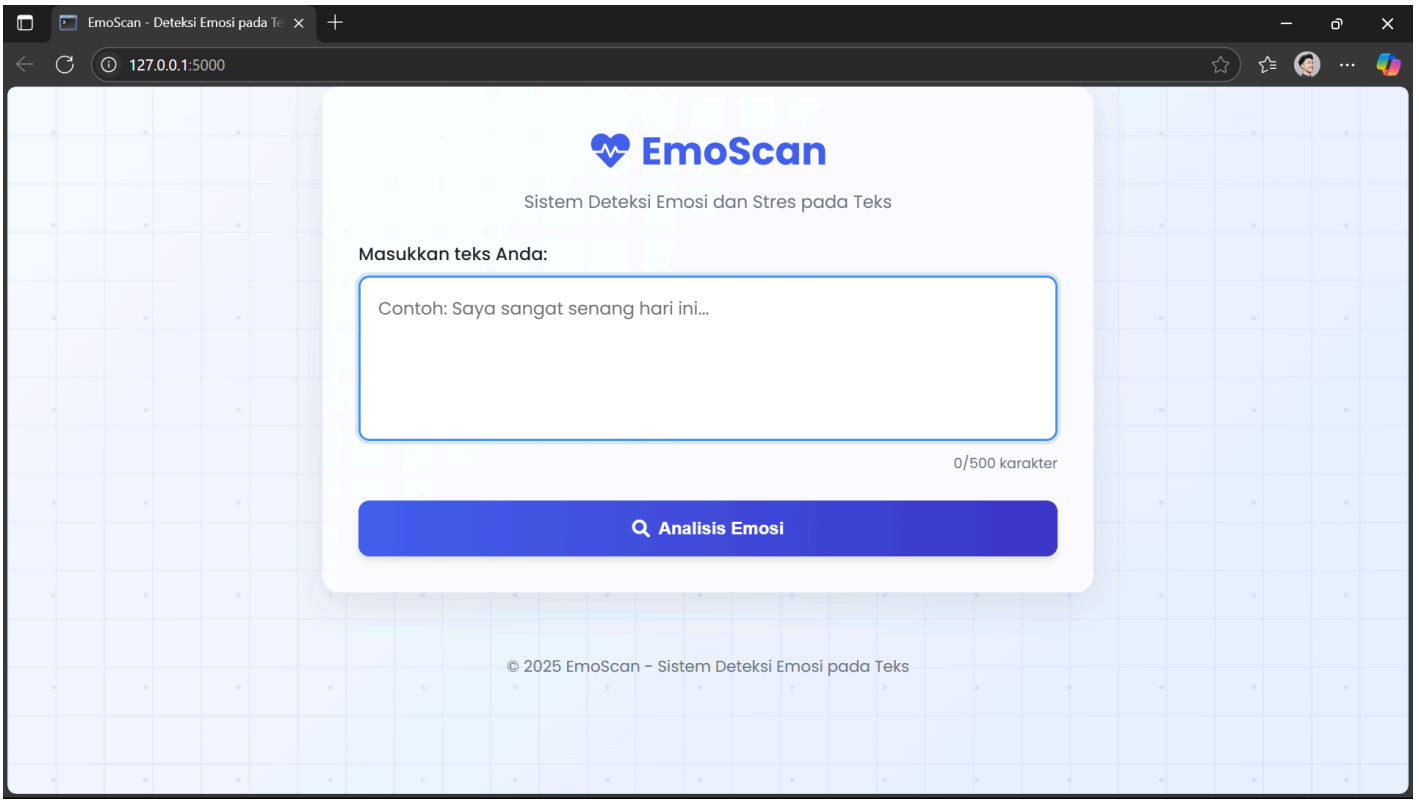


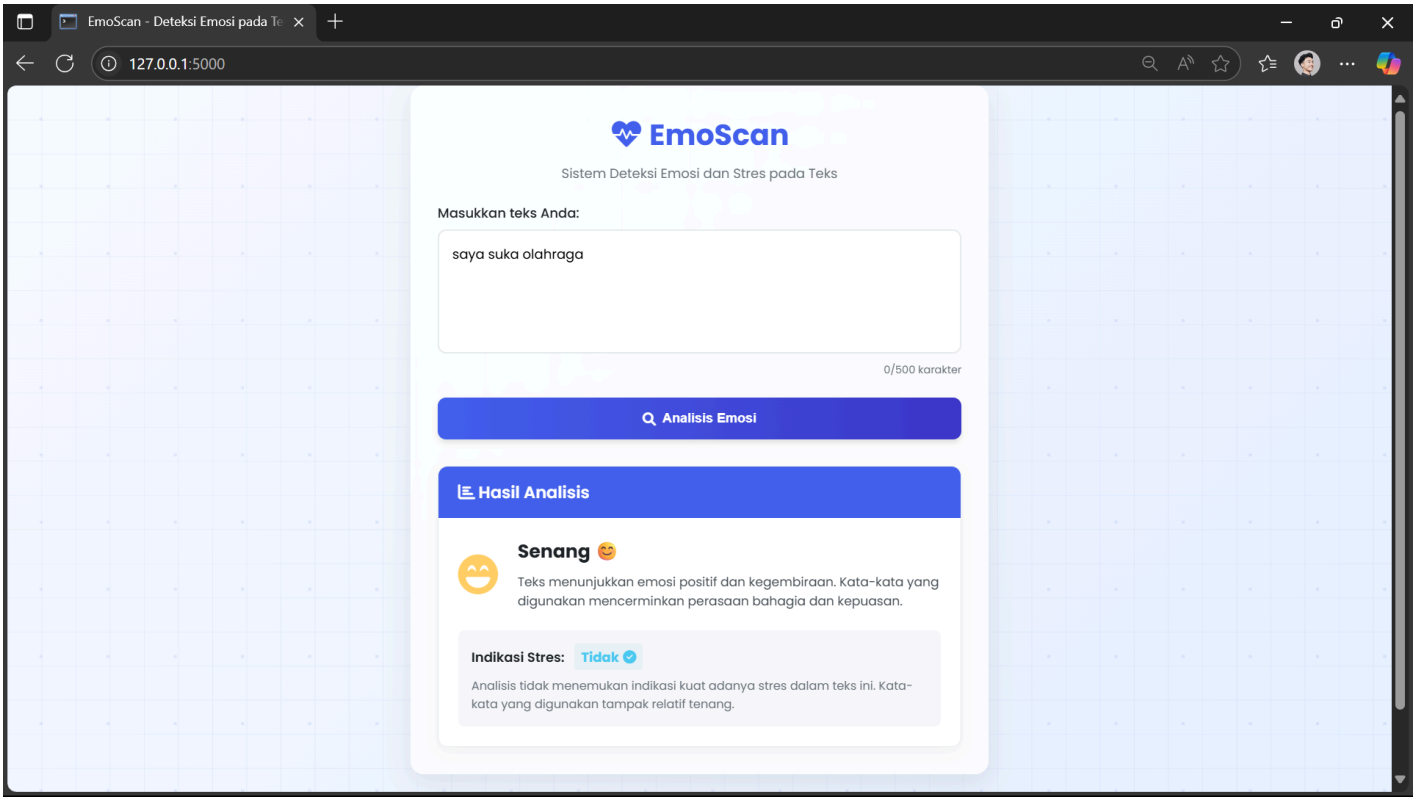
Gambar 3.3 Visualisasi Confusion Matrix

3.8. Implementasi Website

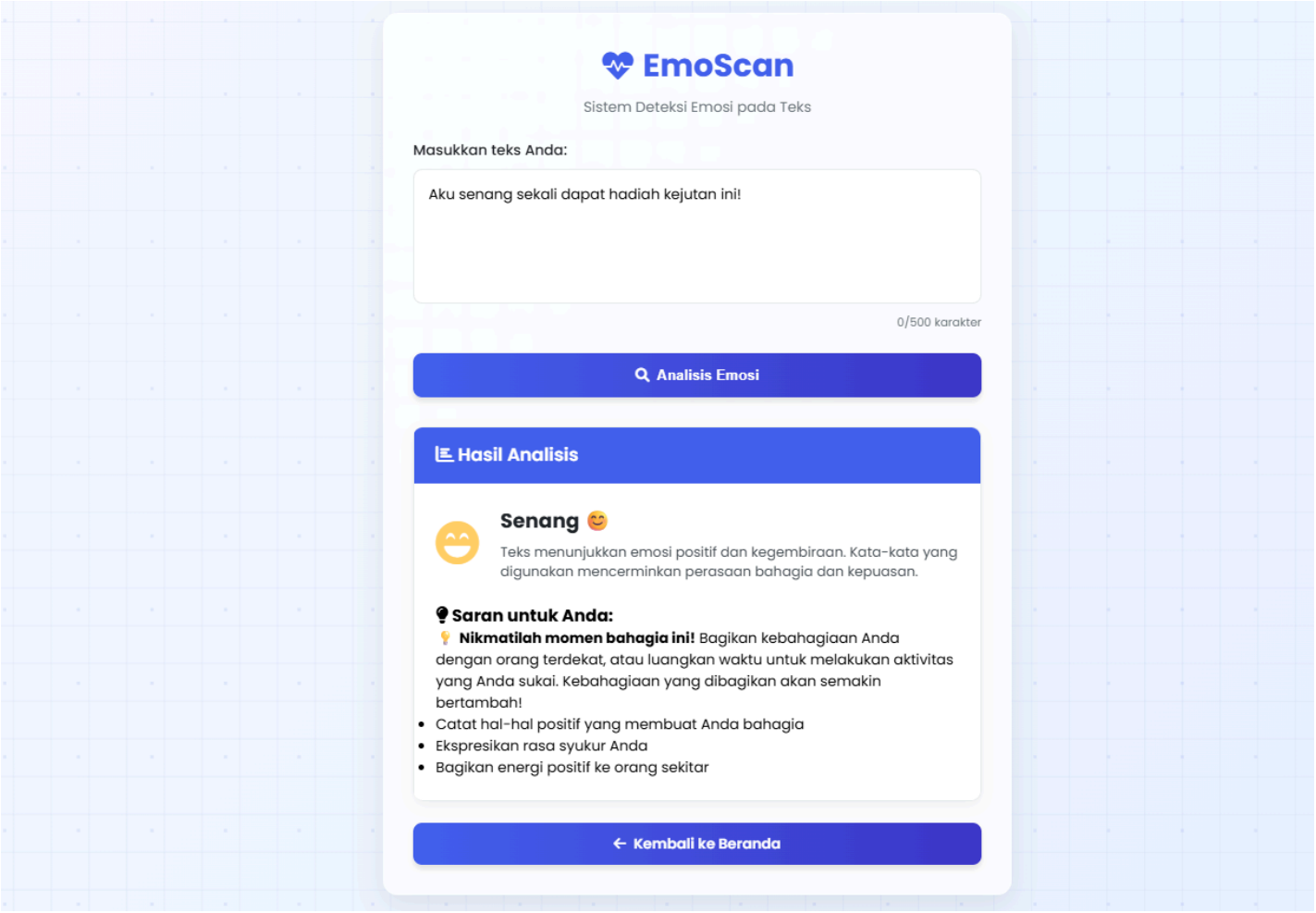


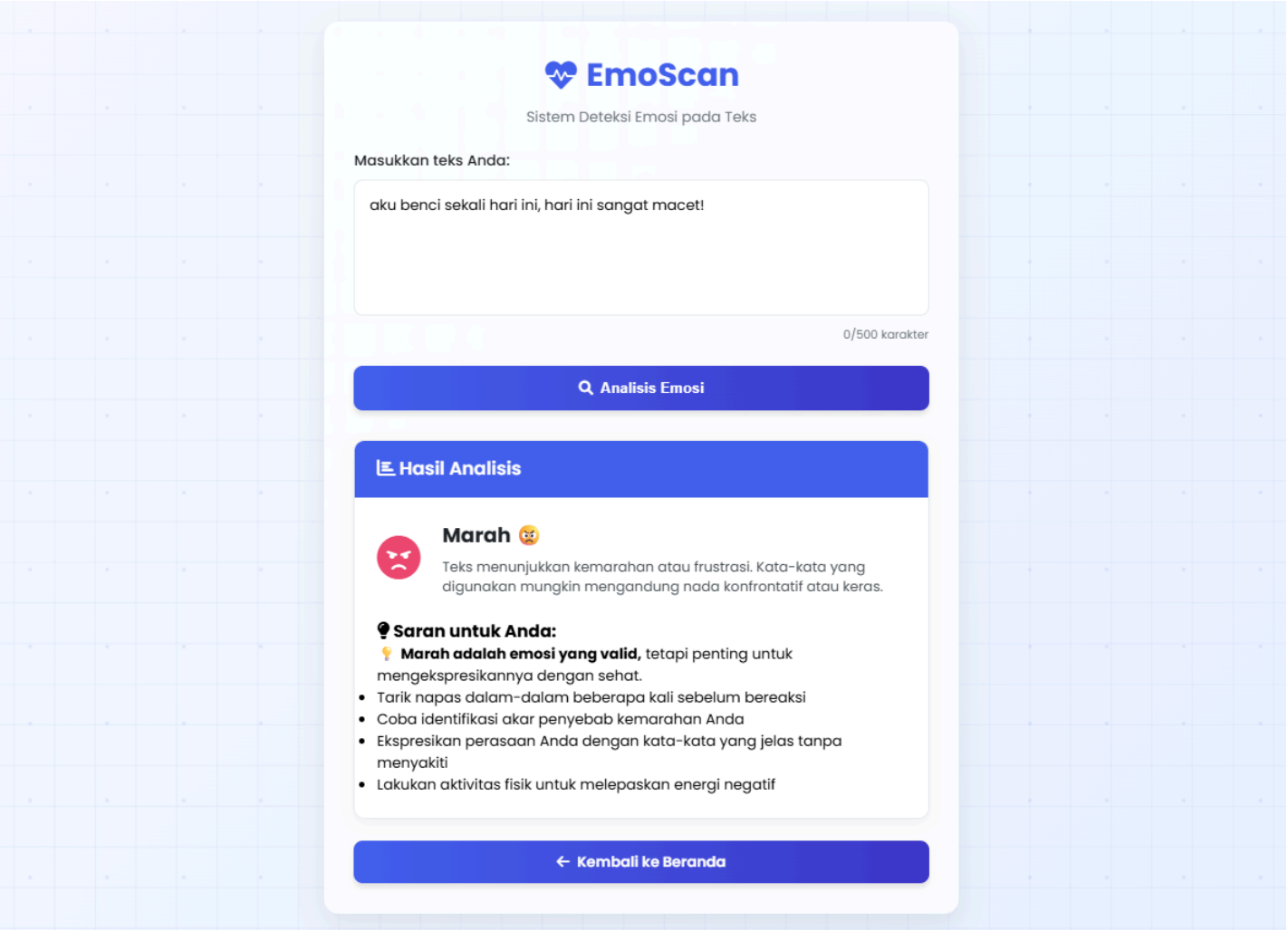
Analisa : Tampilan website pada gambar yang Anda lampirkan sudah terlihat rapi, bersih, dan modern dengan nuansa biru muda yang menenangkan, cocok untuk konteks aplikasi deteksi emosi. Elemen-elemen seperti judul, input teks, tombol aksi, dan hasil deteksi ditampilkan secara terstruktur dalam satu kolom tengah yang responsif, serta penggunaan warna biru pada tombol dan kotak hasil memberikan kesan profesional. Font yang digunakan mudah dibaca, dan penggunaan emoji pada hasil emosi menambah sentuhan humanis yang ramah pengguna. Secara keseluruhan, desain ini sudah sangat layak digunakan dan enak dipandang.





Analisa : Tampilan baru EmoScan tetap mempertahankan kesan bersih dan profesional dengan dominasi putih dan aksen biru, namun kini lebih minimalis dan fokus pada fungsionalitas. Perubahan utama terlihat pada penyederhanaan layout dengan menghilangkan gradien biru muda yang sebelumnya ada, digantikan dengan background putih polos yang membuat konten lebih menonjol. Fitur baru seperti penghitung karakter (0/500) ditambahkan untuk membantu pengguna, sementara hasil analisis emosi dan stres kini ditampilkan lebih terstruktur dengan penjelasan deskriptif yang lebih rinci. Meski beberapa elemen seperti emoji dihilangkan, tampilan baru ini terasa lebih ringkas dan langsung pada inti, dengan hierarki informasi yang lebih jelas sehingga pengguna bisa lebih fokus memahami hasil deteksi emosi mereka.


EmoScan
Sistem Deteksi Emosi pada Teks
Masukkan teks Anda:
Aku senang sekali dapat hadiah kejutan ini!
0/500 karakter
Analisis Emosi
Hasil Analisis
Senang
Teks menunjukkan emosi positif dan kegembiraan. Kata-kata yang digunakan mencerminkan perasaan bahagia dan kepuasan.
Saran untuk Anda:
Nikmatilah momen bahagia ini! Bagikan kebahagiaan Anda dengan orang terdekat, atau luangkan waktu untuk melakukan aktivitas yang Anda sukai. Kebahagiaan yang dibagikan akan semakin bertambah!
Catat hal-hal positif yang membuat Anda bahagia
Ekspresikan rasa syukur Anda
Bagikan energi positif ke orang sekitar
Kembali ke Beranda


EmoScan
Sistem Deteksi Emosi pada Teks
Masukkan teks Anda:
aku benci sekali hari ini, hari ini sangat macet!
0/500 karakter
Analisis Emosi
Hasil Analisis
Marah
Teks menunjukkan kemarahan atau frustrasi. Kata-kata yang digunakan mungkin mengandung nada konfrontatif atau keras.
Saran untuk Anda:
Marah adalah emosi yang valid, tetapi penting untuk mengekspresikannya dengan sehat.
Tarik napas dalam-dalam beberapa kali sebelum bereaksi
Coba identifikasi akar penyebab kemarahan Anda
Ekspresikan perasaan Anda dengan kata-kata yang jelas tanpa menyakiti
Lakukan aktivitas fisik untuk melepaskan energi negatif
Kembali ke Beranda

References

[1] J. Bata, Suyoto, and Pranowo, “Leksikon untuk deteksi emosi dari teks Bahasa Indonesia,” in Proceedings of the Seminar Nasional Informatika (semnasIF 2015), UPN “Veteran” Yogyakarta, pp. 195–202, Nov. 14, 2015.

[2] R. N. S. Amriza and D. Supriyadi, “Komparasi metode machine learning dan deep learning untuk deteksi emosi pada teks di sosial media,” Jurnal JUPITER, vol. 13, no. 2, pp. 130–139, Oct. 2021.

[3] J. Bollen, H. Mao, and A. Pepe, “Modeling public mood and emotion: Twitter sentiment and socio-economic phenomena,” in Proceedings of the International AAAI Conference on Web and Social Media, vol. 5, no. 1, pp. 450–453, 2011.

[4] R. Ullah, N. Amblee, W. Kim, and H. Lee, “From valence to emotions: Exploring the distribution of emotions in online product reviews,” Decision Support Systems, vol. 87, pp. 41–53, 2016.

[5] M. Thelwall, K. Buckley, and G. Paltoglou, “Sentiment in Twitter events,” Journal of the American Society for Information Science and Technology, vol. 62, no. 2, pp. 406–418, 2011.

[6] D. Al-Hajjar and A. Z. Syed, “Applying sentiment and emotion analysis on brand tweets for digital marketing,” in Proc. Applied Electrical Engineering and Computing Technologies (AEECT), IEEE, 2015.

[7] R. Chitturi, R. Raghunathan, and V. Mahajan, “Form versus function: How the intensities of specific emotions evoked in functional versus hedonic trade-offs mediate product preferences,” Journal of Marketing Research, vol. 44, no. 4, pp. 702–714, 2007.

[8] P. V. Marsden and K. E. Campbell, “Reflections on conceptualizing and measuring tie strength,” Social Forces, vol. 91, no. 1, pp. 17–23, 2012.

[9] E. Gilbert and K. Karahalios, “Widespread worry and the stock market,” in Proc. AAAI Conference on Web and Social Media, pp. 59–65, 2010.

[10] P. Burnap and M. L. Williams, “Cyber hate speech on Twitter: An application of machine classification and statistical modeling for policy and decision making,” Policy & Internet, vol. 7, no. 2, pp. 223–242, 2015.

[11] R. van der Zanden, K. Curie, M. van Londen, J. Kramer, G. Steen, and P. Cuijpers, “Web-based depression treatment: Associations of clients’ word use with adherence and outcome,” Journal of Affective Disorders, vol. 160, pp. 10–13, 2014.

[12] P. Rodríguez, A. Ortigosa, and R. M. Carro, “Extracting emotions from texts in e-learning environments,” in Proc. International Conference on Complex, Intelligent, and Software Intensive Systems (CISIS), IEEE, pp. 887–892, 2012.

[13] T. Danisman and A. Alpkocak, “Feeler: Emotion classification of text using vector space model,” in AISB 2008 Convention Communication, Interaction and Social Intelligence, vol. 1, pp. 53–59, 2008.

[14] S. Chaffar and D. Inkpen, “Using a heterogeneous dataset for emotion analysis in text,” in Proc. Canadian Conference on Artificial Intelligence, pp. 62–67, 2011.

[15] K. Sailunaz and R. Alhajj, “Emotion and sentiment analysis from Twitter text,” Journal of Computational Science, vol. 36, pp. 101003, 2019.

FINAL RESULT

Kode Program

```
# Install terlebih dahulu jika diperlukan (di Google Colab)
# !pip install tensorflow sastrawi nltk

import re
import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import classification_report
import pickle
import nltk
nltk.download('stopwords')
from nltk.corpus import stopwords
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
```

Analisa : Kode program ini dimulai dengan mengimpor berbagai library penting untuk pemrosesan data dan pembelajaran mesin, termasuk numpy, pandas, tensorflow, sklearn, dan nltk yang akan digunakan untuk membangun model neural network dan melakukan analisis teks. Selain itu, kode ini juga mengunduh data stopwords dari NLTK, yang akan membantu dalam membersihkan teks dengan menghilangkan kata-kata umum yang tidak memberikan makna.

```
[ ] # Load data
    df = pd.read_csv("Twitter_Emotion_Dataset.csv")

    # Preprocessing teks
    stop_words = set(stopwords.words("indonesian"))

    def clean_text(text):
        text = text.lower()
        text = re.sub(r"http\S+|@\w+|#\w+|\d+|[^\w\s]", "", text)
        text = re.sub(r"\s+", " ", text).strip()
        text = " ".join([word for word in text.split() if word not in stop_words])
        return text

    df["clean_text"] = df["tweet"].astype(str).apply(clean_text)
```

Analisa :Kode ini melanjutkan proses dengan memuat dataset dari file CSV bernama Twitter_Emotion_Dataset.csv menggunakan pandas, yang menghasilkan DataFrame bernama df. Selanjutnya, kode ini mendefinisikan stopwords dalam bahasa Indonesia dan membuat fungsi clean_text() untuk membersihkan teks. Fungsi ini mengubah semua karakter dalam teks menjadi huruf kecil dan menghapus karakter yang tidak diinginkan, seperti angka, simbol, dan whitespace yang berlebihan. Setelah itu, fungsi ini memisahkan teks menjadi kata-kata dan menghilangkan kata-kata yang terdaftar sebagai stopwords. Akhirnya,

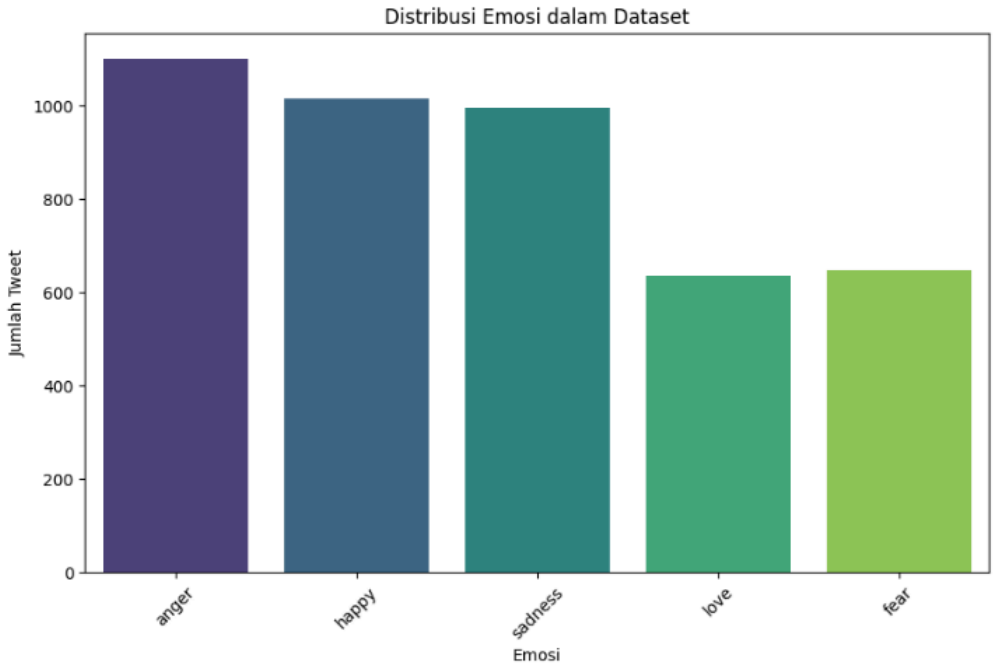
kode ini menerapkan fungsi clean_text() pada kolom tweet dalam DataFrame, menghasilkan kolom baru yang berisi teks yang telah dibersihkan.

```
# 1. Analisis Distribusi Label
plt.figure(figsize=(10,6))
sns.countplot(data=df, x='label', palette='viridis')
plt.title('Distribusi Emosi dalam Dataset')
plt.xlabel('Emosi')
plt.ylabel('Jumlah Tweet')
plt.xticks(rotation=45)
plt.show()
```

```
/tmp/ipython-input-14-1516326575.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `lege

  sns.countplot(data=df, x='label', palette='viridis')
```



Analisa : Pada bagian kode ini, analisis distribusi label dilakukan dengan menggunakan seaborn dan matplotlib. Diagram batang menunjukkan jumlah tweet untuk setiap emosi dalam dataset, termasuk emosi seperti "anger," "happy," "sadness," "love," dan "fear."

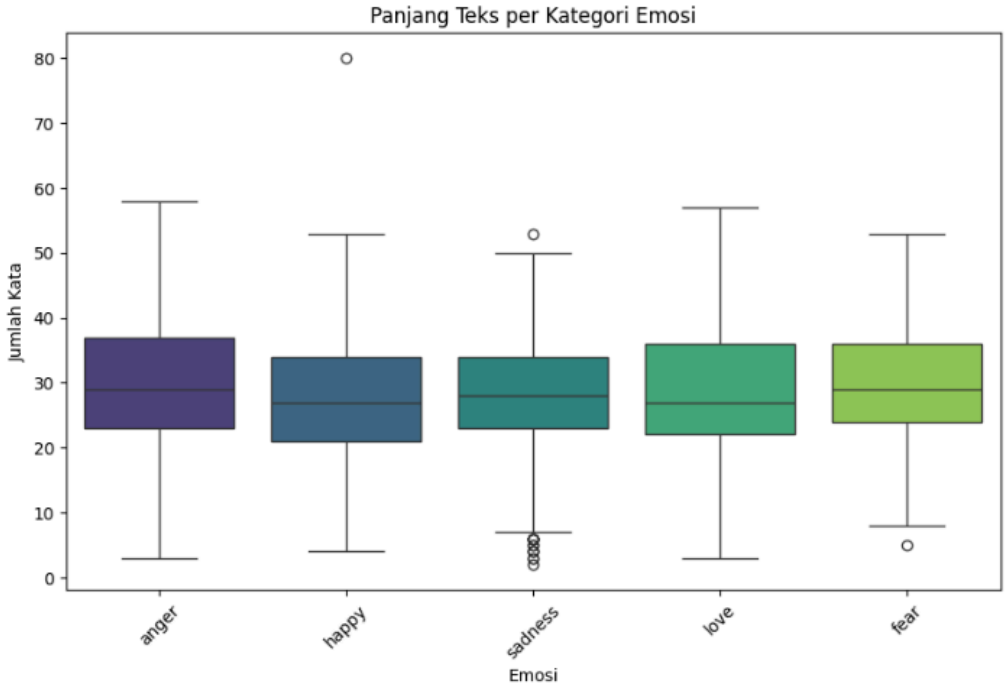
Diagram menunjukkan bahwa emosi "anger" dan "happy" memiliki jumlah tweet yang paling tinggi, menunjukkan bahwa banyak orang mengungkapkan kemarahan dan kebahagiaan dalam teks mereka. Emosi "sadness" juga cukup signifikan, menandakan bahwa kesedihan sering diekspresikan. Sementara itu, "love" dan "fear" terlihat lebih jarang, menunjukkan emosi ini kurang umum diungkapkan dalam dataset.

```
# 2. Analisis Panjang Teks
df['text_length'] = df['tweet'].apply(lambda x: len(x.split()))
plt.figure(figsize=(10,6))
sns.boxplot(data=df, x='label', y='text_length', palette='viridis')
plt.title('Panjang Teks per Kategori Emosi')
plt.xlabel('Emosi')
plt.ylabel('Jumlah Kata')
plt.xticks(rotation=45)
plt.show()
```

```
/tmp/ipython-input-15-179084769.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

  sns.boxplot(data=df, x='label', y='text_length', palette='viridis')
```



Analisa : Kode ini menggunakan pustaka seaborn dan matplotlib untuk memvisualisasikan panjang teks berdasarkan emosi dalam dataset. Pertama, kode menghitung panjang setiap tweet dengan menerapkan fungsi yang membagi teks berdasarkan spasi. Kemudian, diagram kotak (box plot) dibuat untuk masing-masing kategori emosi—"anger," "happy," "sadness," "love," dan "fear."

Selanjutnya, diagram ini menampilkan rentang panjang teks, dengan garis di tengah kotak mewakili median. Variasi panjang teks terlihat jelas; beberapa emosi memiliki rentang yang lebih lebar, menunjukkan kebebasan pengguna dalam mengekspresikan emosi, sementara yang lain lebih terfokus. Titik di luar batas kotak menunjukkan outlier, yaitu panjang teks yang jauh lebih pendek atau lebih panjang dari kebanyakan.

```
# 3. WordCloud untuk Setiap Emosi
def generate_wordcloud(text, title):
    wordcloud = WordCloud(width=800, height=400,
                          background_color='white',
                          stopwords=stopwords.words('indonesian')).generate(text)
    plt.figure(figsize=(10,5))
    plt.imshow(wordcloud)
    plt.title(title, size=15)
    plt.axis('off')
    plt.show()

for emotion in df['label'].unique():
    text = ' '.join(df[df['label']==emotion]['tweet'].tolist())
    generate_wordcloud(text, f'Kata yang Sering Muncul - Emosi: {emotion}')
```



Analisa : Kode ini menghasilkan visualisasi word cloud yang menunjukkan kata-kata yang paling sering muncul dalam kategori emosi tertentu, yaitu "happy" dan "sadness." Fungsi wordcloud digunakan untuk membuat gambaran visual dari kata-kata, dengan ukuran kata mencerminkan frekuensi kemunculannya.

Dalam prosesnya, kode pertama-tama mengimpor pustaka yang diperlukan dan mendefinisikan fungsi untuk menghasilkan word cloud. Setelah itu, data untuk emosi "happy" dan "sadness" diambil, dan kata-kata diolah untuk menghapus stopwords, sehingga hanya kata-kata yang signifikan yang ditampilkan.

Hasilnya adalah visualisasi yang dinamis, di mana kata-kata yang lebih besar menunjukkan frekuensi yang lebih tinggi dalam teks. Analisis ini memberikan wawasan tentang tema umum dan ekspresi emosi dalam tweet, mengindikasikan bagaimana pengguna berkomunikasi tentang kebahagiaan dan kesedihan.

```
[ ]  # Label Encoding
     le = LabelEncoder()
     df["label_enc"] = le.fit_transform(df["label"])

     # Tokenizer dan Padding
     tokenizer = Tokenizer(num_words=5000, oov_token="<OOV>")
     tokenizer.fit_on_texts(df["clean_text"])
     sequences = tokenizer.texts_to_sequences(df["clean_text"])
     padded = pad_sequences(sequences, maxlen=100)

     # Split
     X_train, X_test, y_train, y_test = train_test_split(
         padded, df["label_enc"], test_size=0.2, stratify=df["label_enc"], random_state=42
     )
```

Analisa : Kode ini melanjutkan dengan melakukan label encoding pada kolom label untuk mengubah label kategori menjadi angka. Selanjutnya, Tokenizer digunakan untuk mengonversi teks yang sudah dibersihkan menjadi urutan angka, hanya mempertimbangkan 5000 kata paling umum. Setelah itu, hasilnya dipadding agar semua urutan memiliki panjang yang sama. Terakhir, data dibagi menjadi set pelatihan dan pengujian dengan proporsi 80% untuk pelatihan dan 20% untuk pengujian, memastikan distribusi label yang seimbang.

```
[ ]  model = Sequential([
         Embedding(input_dim=5000, output_dim=128, input_length=100),
         LSTM(64, return_sequences=False),
         Dropout(0.5),
         Dense(32, activation='relu'),
         Dense(len(le.classes_), activation='softmax')
     ])

     model.compile(loss='sparse_categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
     model.summary()

     # Training
     model.fit(X_train, y_train, epochs=6, batch_size=32, validation_data=(X_test, y_test))
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/embedding.py:90: UserWarning: Argument `input_length` is deprecated. Ju
  warnings.warn(
Model: "sequential"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
lstm (LSTM)	?	0 (unbuilt)
dropout (Dropout)	?	0
dense (Dense)	?	0 (unbuilt)
dense_1 (Dense)	?	0 (unbuilt)

```
 Total params: 0 (0.00 B)
 Trainable params: 0 (0.00 B)
 Non-trainable params: 0 (0.00 B)
Epoch 1/6
110/110 ━━━━━━━━━━━━━━━━━━━━ 13s 84ms/step - accuracy: 0.2480 - loss: 1.5744 - val_accuracy: 0.4711 - val_loss: 1.3640
Epoch 2/6
110/110 ━━━━━━━━━━━━━━━━━━━━ 10s 84ms/step - accuracy: 0.5453 - loss: 1.1688 - val_accuracy: 0.5812 - val_loss: 1.0859
Epoch 3/6
110/110 ━━━━━━━━━━━━━━━━━━━━ 9s 73ms/step - accuracy: 0.7782 - loss: 0.6415 - val_accuracy: 0.5516 - val_loss: 1.1695
Epoch 4/6
110/110 ━━━━━━━━━━━━━━━━━━━━ 9s 79ms/step - accuracy: 0.8915 - loss: 0.3791 - val_accuracy: 0.5982 - val_loss: 1.4823
Epoch 5/6
110/110 ━━━━━━━━━━━━━━━━━━━━ 9s 79ms/step - accuracy: 0.9471 - loss: 0.1887 - val_accuracy: 0.6016 - val_loss: 1.3996
Epoch 6/6
110/110 ━━━━━━━━━━━━━━━━━━━━ 8s 69ms/step - accuracy: 0.9625 - loss: 0.1341 - val_accuracy: 0.5812 - val_loss: 1.7006
<keras.src.callbacks.history.History at 0x7ff010c880d0>
```

Analisa : Pada bagian kode ini, model neural network dibangun menggunakan arsitektur Sequential. Model ini dimulai dengan layer embedding yang mengonversi input menjadi representasi vektor, diikuti oleh layer LSTM yang membantu dalam menangkap pola dalam data sekuensial. Layer dropout ditambahkan untuk mencegah overfitting dengan mengabaikan beberapa neuron selama pelatihan. Model kemudian dikompilasi menggunakan loss function sparse_categorical_crossentropy, optimizer adam, dan mengukur akurasi sebagai metrik. Setelah itu, model dilatih dengan data yang telah dipersiapkan. Tabel yang ditampilkan merinci

struktur model, termasuk jumlah parameter di setiap layer. Output menunjukkan akurasi dan loss untuk data pelatihan dan validasi,

```
y_pred = model.predict(X_test)
y_pred_classes = np.argmax(y_pred, axis=1)

print(classification_report(y_test, y_pred_classes, target_names=le.classes_))
```

```
28/28 ━━━━━━━━━━━━━━━━━━━━ 2s 42ms/step
              precision    recall  f1-score   support

       anger       0.64      0.67      0.66       220
        fear       0.72      0.67      0.69       130
       happy       0.54      0.50      0.52       204
        love       0.72      0.65      0.69       127
     sadness       0.40      0.46      0.43       200

    accuracy                           0.58       881
   macro avg       0.61      0.59      0.60       881
weighted avg       0.59      0.58      0.58       881
```

Analisa : Pada bagian kode ini, model yang telah dilatih digunakan untuk melakukan prediksi pada set pengujian (X_test). Hasil prediksi disimpan dalam y_pred, dan kemudian np.argmax digunakan untuk mendapatkan kelas dengan skor tertinggi dari hasil prediksi. Selanjutnya, fungsi classification_report digunakan untuk menghasilkan metrik evaluasi yang meliputi precision, recall, dan f1-score untuk masing-masing kelas emosi yang ada dalam dataset.

Output dari report ini memberikan gambaran komprehensif mengenai kinerja model, menunjukkan bagaimana model dapat mengenali berbagai emosi seperti "anger", "fear", "happy", "love", dan "sadness". Nilai precision dan recall yang ditampilkan menunjukkan seberapa baik model dalam mengklasifikasikan setiap kelas, sementara f1-score memberikan ukuran keseimbangan antara precision dan recall.

```
# Simpan model
model.save("lstm_emotion_model.h5")

# Simpan tokenizer dan label encoder
with open("tokenizer.pkl", "wb") as f:
    pickle.dump(tokenizer, f)

with open("label_encoder.pkl", "wb") as f:
    pickle.dump(le, f)
# Simpan model
model.save("lstm_emotion_model.h5")

# Simpan tokenizer dan label encoder
with open("tokenizer.pkl", "wb") as f:
    pickle.dump(tokenizer, f)

with open("label_encoder.pkl", "wb") as f:
    pickle.dump(le, f)
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. Th
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. Th
```

Analisa : Kode ini menyimpan model LSTM yang telah dilatih dalam format HDF5 menggunakan `model.save`, sehingga bisa digunakan kembali di masa depan. Selain itu, tokenizer dan label encoder juga disimpan dengan `pickle`, memungkinkan pemrosesan data tanpa perlu melatih ulang.

```
[ ] def predict_emotion(text):
        clean = clean_text(text)
        seq = tokenizer.texts_to_sequences([clean])
        pad = pad_sequences(seq, maxlen=100)
        pred = model.predict(pad)
        label = le.inverse_transform([np.argmax(pred)])
        return label[0]

    # Contoh Uji
    print(predict_emotion("Aku sangat bahagia hari ini"))
    print(predict_emotion("Saya kecewa dan sedih"))
    print(predict_emotion("Saya takut dengan situasi ini"))
    print(predict_emotion("Aku marah sekali"))
    print(predict_emotion("Aku sayang kamu"))
    print(predict_emotion("Aku merasa banyak senyum hari ini"))
    print(predict_emotion("Aku suka melihat kamu"))
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 51ms/step
happy
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 50ms/step
sadness
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 50ms/step
fear
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 50ms/step
anger
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 54ms/step
love
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 55ms/step
love
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 60ms/step
love
```

Analisa : Pada bagian kode ini, fungsi predict_emotion(text) didefinisikan untuk memprediksi emosi dari teks yang diberikan. Fungsi ini pertama-tama membersihkan teks menggunakan fungsi clean_text(), lalu mengonversi teks tersebut menjadi urutan angka menggunakan tokenizer. Setelah itu, urutan angka dipadding agar memiliki panjang yang sesuai, sebelum dimasukkan ke dalam model untuk prediksi. Hasil prediksi kemudian diubah kembali ke label asli menggunakan inverse_transform.

Di bagian bawah, beberapa contoh teks diuji menggunakan fungsi ini, dan hasil prediksi emosi ditampilkan. Output menunjukkan emosi yang diprediksi untuk setiap input, seperti "happy," "sadness," dan emosi lainnya.